TECHNICAL ARCHITECTURE & SOURCE CODE REFERENCE

# Security Sentinel Ultra v3.2 (Portable EDR)

เอกสารสรุปโครงสร้าง Source Code, เครื่องมือ (Tools), ไลบรารี และ Windows Native APIs ที่ใช้ในการพัฒนาระบบ

**ชื่อโปรเจกต์:** Security Sentinel Ultra (Tier 1–5) | **แพลตฟอร์ม:** Windows 10 / 11 (Zero-Dependency) | **ตำแหน่งระบบ:** D:\Sec\SecuritySentinel_Portable | **รุ่น:** v3.2 Telemetry Edition

## 1. สรุปเครื่องมือและเทคโนโลยีที่ใช้ในการพัฒนา (Tools & Technology Stack)

**Security Sentinel Ultra** ออกแบบด้วยแนวคิด **Native Zero-Dependency (Living-off-the-System Defense)** โดยใช้คอมไพเลอร์และคลาสมาตรฐานที่ติดตั้งมาพร้อมกับ Windows 10 / 11 ทั้งหมด 100% ไม่ต้องติดตั้ง Python, Node.js หรือ .NET SDK เพิ่มเติม สามารถก๊อปปี้ใส่ Flash Drive ไปใช้งานได้ทันทีทุกเครื่อง:

| หมวดหมู่ (Layer) | เครื่องมือ / คลาส / API ที่ใช้ | หน้าที่และการทำงานในระบบ |
|---|---|---|
| **1. Core Engine & Runtime** | `Windows PowerShell 5.1`<br>`.NET CLR 4.8 Runtime` | เป็นเอนจินหลักในการรันระบบเบื้องหลัง (Daemon), จัดการ Event Loop, อ่าน/เขียน JSON และเชื่อมต่อกับระบบปฏิบัติการ |
| **2. GUI & Visual Frontend** | `WPF (XAML Markup)`<br>`PresentationFramework`<br>`System.Windows.Threading` | สร้างหน้าต่าง Command Center แบบ Modern Dark Cyber UI, หลอดวัด CPU/RAM แบบเรียลไทม์ และอัปเดตหน้าจอผ่าน `DispatcherTimer` ทุก 2 วินาทีโดยไม่บล็อก UI Thread |
| **3. Kernel & Native Process API** | `C# Inline Compiler (Add-Type)`<br>`ntdll.dll!NtSuspendProcess`<br>`ntdll.dll!NtResumeProcess`<br>`kernel32.dll!OpenProcess` | คอมไพล์โค้ดภาษา C# ในหน่วยความจำเพื่อเรียกฟังก์ชันระดับเคอร์เนลของ Windows สำหรับแช่แข็ง (Micro-Freeze) โปรเซสมัลแวร์ทันทีในเสี้ยววินาที (Tier 3) |
| **4. Real-Time File System Guard** | `System.IO.FileSystemWatcher`<br>`Register-ObjectEvent` | ดักจับเหตุการณ์การสร้าง/เปลี่ยนชื่อ/ลบไฟล์ในระดับ Event-Driven (ไม่กิน CPU) ทั้งในโฟลเดอร์เบราว์เซอร์ (Chrome, Edge, Brave, Opera) และไฟล์กับดัก Canary Honeypot |
| **5. Process & Hardware Telemetry** | `CIM / WMI (Win32_Process)`<br>`PerformanceCounter (Processor)`<br>`ComputerInfo (Physical RAM)` | ตรวจสอบการแตกหน่อของโปรเซส (Parent-Child Process Anomaly), ดึงค่า CommandLine เพื่อตรวจจับคำสั่งเข้ารหัสอำพราง และวัดโหลด CPU / RAM ของเครื่องแบบสดๆ |
| **6. Cryptography & Trust Verification** | `Get-AuthenticodeSignature`<br>`Get-FileHash (SHA-256)` | ตรวจสอบใบรับรองดิจิทัล (Digital Signature) ของไฟล์ `.dll` และ `.exe` เพื่อแยกแยะไฟล์ระบบแท้ของ Google/Microsoft ออกจากมัลแวร์ และคำนวณแฮช SHA-256 |
| **7. OS Hardening & Persistence** | `Task Scheduler API`<br>`Windows Registry Providers`<br>`LSASS RunAsPPL & Event 4104` | ลงทะเบียน Auto-Start ตอนเปิดเครื่อง, เฝ้าระวัง Registry 28+ จุด, เปิดบันทึก Script Block Logging (Event ID 4104) และป้องกันการดัมป์รหัสผ่านจาก `lsass.exe` |
| **8. Alerting & Threat Intel** | `Discord Webhook REST API`<br>`VirusTotal API v3 / GUI Link`<br>`WinRT ToastNotificationManager` | ส่งการ์ดแจ้งเตือน Rich Embed แยกสีตามความรุนแรงเข้า Discord พร้อมแท็ก `@everyone`, แนบลิงก์ตรวจสอบ SHA-256 บน VirusTotal และเด้งแจ้งเตือนบนหน้าจอ Windows |

## 2. โครงสร้างไฟล์ทั้งหมดของระบบ (Directory & File Structure)

```
D:\Sec\SecuritySentinel_Portable\
├── 1-Click-Install.bat        # คลิกเดียวเพื่อติดตั้ง ตั้งค่า Hardening และเริ่มรันเกราะป้องกันอัตโนมัติ
├── 1-Click-Uninstall.bat      # คลิกเดียวเพื่อถอนการติดตั้งและล้างไฟล์กับดักทั้งหมด
```

```
├── Sentinel-Dashboard.bat     # สคริปต์เปิดหน้าต่าง GUI Command Center Dashboard
└── core\
    ├── config.json            # ไฟล์เก็บค่าตั้งค่าและลิงก์ Discord Webhook (แยกออกจากโค้ดเพื่อความปลอดภัย)
    ├── Install.ps1            # โค้ดติดตั้งระบบ, เปิด Event 4104 / LSASS RunAsPPL, สร้าง Task & Shortcut
    ├── SentinelWatchdog.ps1   # เอนจินหลักที่ทำงานเบื้องหลังตลอด 24 ชม. (ครบทั้ง Tier 1 ถึง Tier 5)
    ├── SentinelDashboard.ps1  # โค้ดหน้าจอ WPF/XAML GUI, มาตรวัด CPU/RAM, ปุ่ม Scan และ Killswitch
    └── Uninstall.ps1          # โค้ดหยุดโปรเซส ลบ Task Scheduler และทำความสะอาดระบบ
```

## 3. เจาะลึกการทำงานของ Source Code ทีละส่วน (Source Code Breakdown)

### 3.1 เอนจินป้องกันเบื้องหลัง: `core\SentinelWatchdog.ps1` หัวใจหลักของระบบ

ทำงานเป็น Daemon เบื้องหลัง (ใช้แรมเพียง ~30 MB และกิน CPU 0.0% ยามปกติ) แบ่งการทำงานออกเป็น 4 โมดูลหลักตามแผนแม่บท Tier 1–5:

**โมดูล 1: C# Native Kernel Control (Tier 3)**

- ใช้ `Add-Type` คอมไพล์คลาส `[SentinelNative.KernelControl]`
- เชื่อมต่อไปยัง `ntdll.dll` เพื่อสร้างฟังก์ชัน `FreezeProcess(int pid)` และ `ThawProcess(int pid)`
- หยุดการทำงานของทุก Thread ในโปรเซสมัลแวร์ทันทีด้วย `NtSuspendProcess`

**โมดูล 2: Authenticode & Safe Quarantine (Tier 1 & 4)**

- ฟังก์ชัน `Test-IsTrustedBinary` ตรวจสอบใบรับรองดิจิทัลของไฟล์ว่าเป็นของ Google, Microsoft, Brave, Opera, Steam หรือไม่
- ฟังก์ชัน `Move-ToQuarantine` มีลูป Retry 6 รอบ (ทุก 250ms) เพื่อแก้ปัญหา File Lock และสร้างไฟล์ `.meta.txt` เก็บค่า SHA-256

**โมดูล 3: Browser Guard & Canary Trap (Tier 1)**

- ผูก `FileSystemWatcher` เฝ้าดูโฟลเดอร์ `User Data` ของ Chrome, Edge, Brave, Opera
- วางไฟล์เหยื่อล่อ (Decoy) ใน `AppData` และ `Documents` เพื่อดักจับมัลแวร์สาย Infostealer และ Ransomware

**โมดูล 4: Process Tree & Webhook Engine (Tier 2 & 5)**

- ฟังก์ชัน `Inspect-NewProcesses` ตรวจจับเมื่อเบราว์เซอร์หรือเกมแอบรัน `powershell.exe`, `cmd.exe`, `mshta.exe`
- ตรวจจับคำสั่ง `-EncodedCommand` / `IEX` และส่งการ์ดแจ้งเตือนพร้อมลิงก์ VirusTotal เข้า Discord Webhook

```
# ตัวอย่างโค้ดสำคัญใน SentinelWatchdog.ps1: C# P/Invoke NtSuspendProcess & Parent-Child Anomaly Detection
[DllImport("ntdll.dll", SetLastError = true)]
private static extern int NtSuspendProcess(IntPtr processHandle);

$isHighRiskParent = ($parentName -match "^(chrome|msedge|brave|opera|nikke|nikke_launcher|discord|telegram|steam)\.exe$")
$isShellOrLolbin  = ($cName -match "^(powershell|pwsh|cmd|mshta|wscript|cscript|regsvr32|certutil|bitsadmin)\.exe$")

if ($isHighRiskParent -and $isShellOrLolbin) {
    if ($script:Config.AutoFreezeSuspiciousProc) {
        [SentinelNative.KernelControl]::FreezeProcess([int]$cPid)
    }
    $intel = Get-FileThreatIntel -FilePath $exe
    Show-Alert -Title "Tier 2: Parent-Child Process Anomaly!" -Level "TIER2" -SHA256 $intel.SHA256 ...
}
```

### 3.2 หน้าจอควบคุมหลัก: `core\SentinelDashboard.ps1` Cyber-SOC GUI v3.2

| ส่วนประกอบบนหน้าจอ (UI Component) | รายละเอียดการทำงานในโค้ด |
|---|---|
| **1. แผงวัดทรัพยากรเครื่อง (Row 1)**<br>**CPU & RAM Telemetry Gauges** | • **System CPU Usage:** อ่านค่าจาก `PerformanceCounter("Processor", "% Processor Time", "_Total")` พร้อมเปลี่ยนสีหลอดตามโหลด (`OPTIMAL` / `MODERATE` / `HIGH LOAD`)<br>• **System Memory (RAM):** คำนวณจาก `TotalPhysicalMemory` และ `AvailablePhysicalMemory` แสดงเป็น GB และ %<br>• **Sentinel Engine Footprint:** คำนวณอัตราการใช้ CPU (%) จากค่าความต่างของ `TotalProcessorTime` และแรมจาก `WorkingSet64` ของโปรเซส Watchdog โดยตรง |
| **2. แผงสถานะ 5 ระดับ (Row 2)**<br>**5-Tier Architecture Matrix** | แสดงสถานะการทำงานของทั้ง 5 Tiers (Core Watchdog, Process Guard, Response Engine, Zero-Trust, และ Discord & VT Intel) โดยตรวจสอบโปรเซสผ่านไฟล์ `sentinel.pid` ที่รวดเร็วและไม่กินรอบซีพียู |

| ส่วนประกอบบนหน้าจอ (UI Component) | รายละเอียดการทำงานในโค้ด |
|---|---|
| **3. แถบตั้งค่า Discord Webhook (Row 3)** | มีช่องกรอก URL (`TxtWebhookUrl`), ตัวเลือก `Ping @everyone`, ตัวเลือก `Auto-Freeze Process`, ปุ่มบันทึกค่าลง `config.json` และปุ่ม **Test Notification** ที่ยิงการ์ดทดสอบพร้อมแนบค่า CPU/RAM ปัจจุบันเข้า Discord ทันที |
| **4. ปุ่มควบคุมฉุกเฉิน (Row 5) Action & Emergency Dock** | • `[KILLSWITCH] Emergency Cut Network`: สั่งตัดเน็ตทันทีด้วย `ipconfig /release` และปิดการ์ดแลน (หากมีสิทธิ์ Admin) พร้อมสลับปุ่มเป็นสีเขียวเพื่อกดต่อเน็ตคืน (`/renew`)<br>• `[SCAN] Deep Threat Scan (Tier 1-5)`: สแกนโฟลเดอร์เบราว์เซอร์, ตรวจสอบโปรเซสที่รันจาก `%TEMP%`, เช็ค CLSID และตรวจสอบสถานะ Hardening |

### 3.3 ตัวติดตั้งและการจัดการคอนฟิก: `core\Install.ps1`, `core\Uninstall.ps1` & `core\config.json`

- **`core\Install.ps1`:** สร้างโฟลเดอร์ใน `%LOCALAPPDATA%\SecuritySentinel`, เปิด **PowerShell Script Block Logging (Event ID 4104)**, เปิด **LSASS Memory Protection (`RunAsPPL = 1`)** เมื่อรันด้วยสิทธิ์ Admin, ลงทะเบียน **Scheduled Task** และสร้างไอคอนลงบนหน้า Desktop ที่รองรับ OneDrive Desktop
- **`core\config.json`:** แยกเก็บค่าตั้งค่าและลิงก์ `DiscordWebhookUrl` ออกจากไฟล์สคริปต์ `.ps1` เพื่อป้องกันไม่ให้ระบบสแกนอัตโนมัติของ Cloud Storage (เช่น Google Drive) เข้าใจผิด (และเมื่อจะแชร์โฟลเดอร์ผ่าน Google Drive แนะนำให้บีบอัดเป็นไฟล์ `.zip` แบบใส่รหัสผ่านเสมอ)